

## CLAUDE.md — Inkglass Odyssey

*Project reference document for Claude Code. Read this before making any changes.*

*Companion file: REFERENCES.md (patterns from existing games).*

*Last updated: May 2026 — Revision 5. Subsystem design documents extracted to docs/.  
This file is the architectural spine; design docs carry the detail.*

---

## Design Document Index

Read the relevant design document **before** working on the listed directories or endpoints.

| Document                  | Read before working on                                    | Covers                                                                                                                                                                                  |
|---------------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| docs/combat_system.md     | relay/combat/,<br>relay/checks/                           | AC, saves, conditions, advantage/disadvantage, damage types, action economy, initiative, death state, passive checks, rest, HP formula, specialisation paths, environmental interaction |
| docs/economy_balance.md   | relay/economy/,<br>item/recipe schemas,<br>shop endpoints | Earning rates, pricing tables, damage/healing scaling, sell-back, transport fares, crafting margins, gathering yields, item types, binding, gear slots                                  |
| docs/companion_system.md  | relay/companions/                                         | Recruitment, combat AI, incapacitation, loyalty strain, ambient behaviour, persistence, reunion                                                                                         |
| docs/faction_system.md    | relay/factions/, shop pricing                             | Standing tiers, propagation, effects, schema                                                                                                                                            |
| docs/content_authoring.md | NPC files, item files, lore, recipes, world events        | Authoring quality priorities, balance guidance, lore chunk quality, Workshop form structure                                                                                             |
|                           |                                                           |                                                                                                                                                                                         |

|                               |                                            |                                                                                                     |
|-------------------------------|--------------------------------------------|-----------------------------------------------------------------------------------------------------|
| docs/narrative_control.md     | relay/narrative/,<br>relay/admin/          | Admin/DM/Solo interfaces,<br>content override model, plot<br>beats, world reset, admin<br>interface |
| docs/schemas_reference.md     | /schemas/,<br>relay/schemas.py             | Complete field definitions for all<br>JSON schemas                                                  |
| docs/narrative-control-ui.pdf | Narrative endpoints,<br>Workshop UI        | Canonical spec for narrative<br>control panels and plot beat<br>schema                              |
| docs/prompt_engineering.md    | relay/ai/rp_prompts.py,<br>chat_prompts.py | Prompt structure, few-shot<br>format, check result integration                                      |

# 1. What This Project Is

Inkglass Odyssey is a private, application-gated, multi-world AI-narrative collaborative-fiction RPG. Unity desktop client (Windows/Mac/Linux) communicating with a Python relay service for all persistent state and AI integration. Solo and small-group multiplayer ( $\leq 8$  players, friend-group scope).

**Primary mode:** freeform prose RP between players and AI-driven NPCs. **Secondary mode:** quick-chat for transactional NPCs. **Design references:** SillyTavern, Disco Elysium, Pentiment, BG3, tabletop play-by-post.

**Out of scope:** photorealistic rendering, action combat, public distribution, mobile/console, VR.

## 1.1 Worlds

Seven worlds, two access tiers. Each is open-world with own traversal, economy, progression terms, and content library.

| World ID       | Setting & primary traversal                                           |
|----------------|-----------------------------------------------------------------------|
| inkglass_dark  | Dark fantasy. Tier 1. Ground mounts, watercraft, carriages.           |
| murim          | Wuxia/cultivation. Tier 1. Qinggong, mounts, river boats.             |
| cybernightlife | Neon-saturated urban. Tier 1. Vehicles, public transit, aerial taxis. |
|                |                                                                       |

|               |                                                           |
|---------------|-----------------------------------------------------------|
| wha_au        | Witch Hat Atelier AU. Tier 2. Broom flight, foot, mounts. |
| atla_au       | Avatar AU. Tier 2. Sky companion, gliders.                |
| gachiakuta_au | Gachiakuta AU. Tier 2. Vertical traversal, climbing.      |
| hxxh_au       | Hunter x Hunter AU. Tier 2. Vehicles, ships, blimps.      |

*Fandom AU worlds: renamed locations, renamed mechanics, original characters only. No canon names. The \_au suffix is mandatory.*

## 1.2 Solo and Multiplayer

Both modes share architecture, relay, schemas, and content. Solo: 60–120 min sessions, player-confirmed implicit checks. Multiplayer: 120–240 min, LLM-decided checks for flow. FishNet networking (solo: host-only). Session recovery via pending-turn records.

## 2. Architecture

### 2.1 The Keystone Principle

*The relay is the source of truth. Unity is a view. All persistent state lives in the relay's database. Unity's local state is a cache. Every state change goes through a relay API endpoint. No exceptions.*

### 2.2 Dialogue Mode Architecture

**RP mode:** prose input/response, scene state tracked, implicit checks, long sessions. **Quick-chat:** dialogue-line format, no scene state, explicit checks, short sessions. Never mixed.

#### RP Mode Turn Flow

1. Player writes prose → Unity sends via WebSocket → relay writes pending-turn record
2. Relay constructs prompt (NPC personality + scene state + recent turns + RAG lore + few-shot)
3. First LLM call: structured analysis (checks, scene-state changes, animation directives, draft response)
4. Relay validates all check types and difficulties against registry; out-of-range values clamped

5. Check resolution (solo: player confirmation; multiplayer: automatic)
6. Second LLM call: final prose with check results; streamed to Unity
7. Scene state + NPC memory + animation directives committed atomically; pending-turn complete

Two LLM calls per turn (three with solo check confirmation). Prompt caching on static sections mandatory.

## WebSocket Protocol (WS /dialogue)

JSON frames with type discriminator. Client → relay: rp\_turn, quickchat\_turn, check\_confirm, heartbeat. Relay → client: stream\_start, stream\_chunk, stream\_end, check\_proposal, check\_result, animation\_directive, scene\_update, error, heartbeat\_ack. One message at a time per session; second in-flight turn returns error code turn\_in\_progress.

## 2.3–2.9 Supporting Systems (Brief)

**Session recovery:** pending\_turn with stage markers. RelayClient exponential backoff (1s–30s). GET /session/{id}/state on reconnect.

**Multi-NPC scenes:** shared scene context document per NPC; scene narrator arbitrates turns and NPC-to-NPC dialogue.

**EAS:** prose-derived animation directives validated against world registry before Unity receives them. LLM proposes, EAS validates, Unity executes.

**Prompt caching:** Tier 1 (static, cache aggressively), Tier 2 (session-stable, invalidate on scene changes), Tier 3 (dynamic, never cached). See docs/prompt\_engineering.md.

**Narrative Director:** rules-based pacing agent (not LLM-driven). Evaluates triggers after each turn, writes director\_signal to scene\_state. Scene narrator's next LLM call receives the signal. Director proposes; narrator executes; NPC responds.

**Canon mutation:** post-session diff agent proposes fact changes. Player/DM confirms before commit. Never automatic.

**Semantic RAG:** embedding vectors + keyword fallback, cosine similarity, per-turn token budget.

## 2.10 Tech Stack

**Unity:** 6.1 LTS, URP, C# .NET Standard 2.1, FishNet, UMA 2, UniVRM, Cinemachine,

TextMeshPro.

**Relay:** Python 3.12+, FastAPI (HTTP + WebSocket), SQLAlchemy + SQLite (Postgres migration path), SQLite-vec, Anthropic SDK, HTTPX, Pydantic, Alembic, PyJWT.

TTS deferred to Phase 4 (ADR required for provider selection).

### 3. Game Systems (Summaries)

*Full specifications in the linked design documents. These summaries exist so Claude Code knows the systems exist and where to find detail.*

**Progression (3.2):** D&D-inspired, levels 1–20 in four bands (Newcomer/Established/Known/Legendary). Six canonical ability scores with per-world display names. Level-up at session end via POST /session/end. See docs/combat\_system.md for HP formula and specialisation paths.

**Economy (3.3):** per-world currencies, relay-authoritative wallets. Earning rates, pricing, sell-back, transport fares, and crafting margins defined in docs/economy\_balance.md.

**Items and gear (3.4):** type-specific stats blocks, binding system, rarity-scaled damage/healing/pricing. See docs/economy\_balance.md.

**Flora, fauna, gathering (3.5–3.6):** region-defined, check-gated. Sapient fauna are NPCs (entity\_class: creature). Gathering yields in docs/economy\_balance.md.

**Crafting (3.7):** recipe-driven, check-resolved, profitability-balanced. See docs/economy\_balance.md.

**Shops (3.8):** NPC-operated, faction-price-modified, prerequisite-gated. Haggling via implicit checks.

**Combat (3.9):** AC, attack rolls, saving throws, advantage/disadvantage, damage types with resistance/vulnerability/immunity, conditions, action economy, initiative, death state, passive checks, rest. Full specification in docs/combat\_system.md.

**Companions (3.10):** recruitment scenarios, combat AI with behavior types, incapacitation/loyalty strain, ambient behaviour, session persistence, post-reset reunion. Full specification in docs/companion\_system.md.

**Factions (3.11):** -100 to 100 standing, five tiers, propagation to allied/rival factions. Full specification in docs/faction\_system.md.

**Environmental interaction (3.12):** scene-state modifiers (darkness, terrain, elevation, weather, hazards). See docs/combat\_system.md.

**Narrative control (2.11):** Admin Workshop / DM Workshop / Narrative Control Panel.  
Content override model, plot beats, world reset. See docs/narrative\_control.md and docs/narrative-control-ui.pdf.

## 4. Traversal System

Each world defines traversal modes in world\_config.json. Unity reads at world load. Every mode carries: canonical ID, display name, Unity controller class, access condition, operating cost. Per-world traversal configurations documented in the world-specific sections below this header (retained from v5 — no change to traversal content).

## 5. Repository Structure

inkglass/ |—— CLAUDE.md THIS FILE |—— REFERENCES.md |—— README.md |——  
.gitignore |—— .claude/ | |—— settings.json Hooks and permissions | |—— agents/  
Architect, coder, tester, reviewer | |—— commands/ Slash command library |—— .github/  
| |—— workflows/ | |—— ci.yml CI pipeline (see Section 7.5) |—— docs/ | |——  
combat\_system.md | |—— economy\_balance.md | |—— companion\_system.md | |——  
faction\_system.md | |—— content\_authoring.md | |—— narrative\_control.md | |——  
schemas\_reference.md | |—— narrative-control-ui.pdf | |—— prompt\_engineering.md  
|—— relay/ | |—— main.py | |—— config.py | |—— logging\_config.py Structured JSON  
logging | |—— models.py SQLAlchemy ORM | |—— schemas.py Pydantic — mirrors  
/schemas/\*.json | |—— migrations/ Alembic | |—— ai/ | |—— npcs/ | |—— scenes/  
narrator.py, director.py | |—— canon/ | |—— checks/ | |—— animation/ | |—— world/ |  
|—— economy/ | |—— combat/ resolver.py, initiative.py, conditions.py, death\_state.py |  
|—— companions/ manager.py, combat\_ai.py, ambient.py, loyalty.py | |—— factions/  
reputation.py | |—— narrative/ events.py, plot\_beats.py, room\_state.py | |—— admin/  
app.py (port 8081), static/, reload.py | |—— traversal/ | |—— auth/ tokens.py,  
middleware.py | |—— middleware/ rate\_limit.py | |—— persistence/ | |—— endpoints/ |  
|—— tests/ | |—— pyproject.toml |—— schemas/ | |—— character\_sheet.json | |——  
npc\_personality.json | |—— ability.json | |—— quest.json | |—— canon\_fact.json | |——  
scene\_state.json | |—— world\_config.json | |—— world\_events.json | |——  
animation\_directive.json | |—— item.json | |—— recipe.json | |—— shop\_data.json |  
|—— faction.json | |—— traversal\_mode.json |—— animations/{world\_id}/registry.json  
|—— lore/{world\_id}/ | |—— npcs/{world\_id}/ | |—— items/{world\_id}/ |——  
crafting/{world\_id}/ | |—— abilities/{world\_id}/ | |—— scenarios/{world\_id}/ |——  
quests/{world\_id}/ | |—— regions/{world\_id}/ |—— unity/ Unity project root

## 6. Schema Definitions

All schemas are fully defined in docs/schemas\_reference.md. Pydantic models in relay/schemas.py mirror /schemas/\*.json exactly. CI validates all content files against schemas on every push.

## 7. Code Conventions

### 7.1 Python (Relay)

- ruff format + ruff check — enforced by PostToolUse hook
- Type hints on all public functions (from **future** import annotations)
- Pydantic models mirror JSON schemas exactly
- Async-by-default for all I/O
- One endpoint per file under relay/endpoints/
- Tests via pytest. Mutation testing (mutmut) on relay/checks/, relay/economy/, relay/canon/ — 85%+ score. Other modules: 90%+ line coverage.
- No global state. Dependencies via FastAPI Depends(). DB sessions via get\_db (yields async SQLAlchemy session, commits on success, rolls back on exception).
- Secrets via environment variables only. Never commit .env.
- Rate limiting on all endpoints. WebSocket: max 1 message per 3 seconds per session.
- **Logging:** Python logging module. Logger name matches module path. Levels: ERROR (unrecoverable), WARNING (recoverable anomalies), INFO (request lifecycle), DEBUG (prompt/LLM details). Structured JSON via relay/logging\_config.py. Never log API keys or player/NPC prose at INFO+.
- **Migrations:** Alembic. Every model change needs a migration. Run alembic upgrade head before starting relay. No raw DDL outside Alembic.

### 7.2 C# (Unity)

Standard C# naming. RelayClient is the only HTTP surface. Async/await with cancellation tokens. No persistent state in Unity — relay is canonical.

### 7.3 JSON Content

Filenames snake\_case, match the id field. Lower-case ASCII only in IDs. Validate against

/schemas/ before committing.

## 7.4 NPC Personality File Authoring

Every NPC file must pass the probe suite before merge. Shop/transport NPCs validated against their respective sub-schemas. See docs/content\_authoring.md for quality guidelines.

## 7.5 CI Pipeline (GitHub Actions)

1. JSON schema validation (all content directories)
2. NPC probe suite (voice, knowledge, manipulation resistance, animation)
3. Canon character name blocklist (AU directories)
4. Python lint and format (ruff)
5. Python tests + mutation testing
6. Alembic migration check

## 8. Relay API Surface

### 8.1 Error Response Schema

All errors: { code (string), message (string), turn\_id (optional), narrative\_hint (optional) }.  
HTTP status codes: 400/401/403/404/409/429/500.

### 8.2 Core Endpoints

POST /session/start, POST /session/end (includes level\_increment + canon diff), GET /session/{id}/state, GET/POST/PATCH /character, GET /npc/{id}, WS /dialogue, POST/GET/POST /scene, POST /dice/roll, POST /checks/implicit, POST/GET/POST /combat, GET/PATCH /quest, GET /lore/search, GET/POST/PATCH /canon, GET flora/fauna, POST /gather, GET/POST /shop, POST /craft, GET/PATCH /inventory, POST /traversal, PATCH /player position, GET /analytics.

Full endpoint tables including companion, faction, narrative control, DM workshop, and admin endpoints are in the v5 reference and docs/narrative\_control.md.

## 9. Database Migration



SQLite for Phase 0–1. Migration trigger:  $\geq 3$  concurrent multiplayer players or  $> 200\text{ms}$  write latency. ADR-gated. SQLAlchemy makes no SQLite-specific assumptions; aiosqlite  $\rightarrow$  asyncpg.

## 10. Application Gating and Auth

**Tier 1:** approved application  $\rightarrow$  account token. **Tier 2:** demonstrated good-faith engagement, manually granted by project owner.

**Session tokens:** HS256 JWT (INKGLASS\_JWT\_SECRET), payload: player\_id, world\_id, session\_id, tier, role (player | dm), mode (solo | multiplayer), iat, exp. Account token authenticates POST /session/start  $\rightarrow$  returns session JWT. Validated on every HTTP request and WebSocket message.

**Multiplayer DM:** room creator gets role: "dm". No mid-session transfer.

## 11. Analytics

Per-session metrics: llm\_call\_count (expected: 2 RP standard, 3 with solo check confirmation, 1 quick-chat), turn\_latency\_p95, cache\_hit\_rate, scene\_state\_size\_bytes, session\_duration, player\_turn\_length\_trend, check\_pass\_rate.

## 12. Content Guide

*Sexual content involving characters whose age is ambiguous or who could be interpreted as minors is absolutely prohibited.*

**content\_rating** (moderate | mature) defined per world. Moderate: violence implied/aftermath. Mature: violence with physical detail and consequence — not gratuitous. See docs/content\_authoring.md for full guidelines.

Fandom AU: no canon dialogue, no canon names, no recognisable locations. NPC files must not extract real-world harmful information. Access revocation for harmful content abuse.

## 13. Critical Invariants

| # | Invariant                                                                            |
|---|--------------------------------------------------------------------------------------|
| 1 | Relay is source of truth for all persistent state                                    |
| 2 | Schemas are versioned — breaking changes need ADR + simultaneous relay/client update |

|    |                                                                         |
|----|-------------------------------------------------------------------------|
| 3  | RelayClient is the only HTTP surface in Unity                           |
| 4  | API keys stay in relay — pre-write hook enforces                        |
| 5  | Fandom AU: no canon names                                               |
| 6  | Solo and multiplayer share content                                      |
| 7  | Lore is data, not code                                                  |
| 8  | LLM is never authoritative over mechanical state                        |
| 9  | Every NPC file includes manipulation-resistance examples (CI validates) |
| 10 | RP mode and quick-chat mode are distinct and never mixed                |
| 11 | Animation directives are relay-validated before Unity receives them     |
| 12 | Session state persisted before processing (pending-turn)                |
| 13 | Rate limiting on all endpoints                                          |
| 14 | All economy transactions through relay endpoints                        |
| 15 | Traversal mode validated against world registry                         |
| 16 | Workshop writes are atomic — complete or rollback, no partial state     |
| 17 | DM Workshop writes never propagate to global defaults                   |
| 18 | Character sheets never modified by workshop actions                     |
| 19 | World reset archives before clearing (7-day retention, non-negotiable)  |
| 20 | Companion state persists across sessions                                |
| 21 | Damage type resistances applied after damage roll                       |
| 22 | Passive checks evaluated automatically against hidden elements          |
| 23 | Faction reputation propagates to allies/rivals automatically            |
| 24 | LLM never consulted to validate workshop actions                        |

## 14. What NOT to Do

- Do not store persistent state in Unity.
- Do not write combat, crafting, gathering, or economy logic in Unity.
- Do not let the LLM decide quest completion, item yields, or crafting outcomes.
- Do not trust LLM-proposed check difficulties without validation.
- Do not introduce a second LLM provider without an ADR.
- Do not bypass schema validation on any content file.
- Do not use canon character names in fandom AU files.
- Do not hardcode world/item/NPC/traversal/currency IDs.
- Do not commit secrets.
- Do not skip few-shot examples or animation\_profile in NPC files.
- Do not run canon\_diff\_agent without player/DM confirmation.
- Do not migrate to Postgres without an ADR.
- Do not allow DM Workshop writes to modify global defaults.
- Do not allow solo players to directly edit world-state flags.
- Do not permanently kill companion NPCs on incapacitation.
- Do not stack numerical situational modifiers (use advantage/disadvantage).
- Do not apply conditions without duration tracking.
- Do not expose NPC mechanical stat blocks to the LLM.
- Do not implement TTS, equipment repair, or housing before their phases.
- Do not create hardcoded Workshop forms — schema-driven only.

## 15. Per-Phase Guidance

**Phase 0 (Current):** Relay spine. Logging → Alembic → schemas → models → auth → character endpoints → WebSocket dialogue → session/scene → RAG → combat resolver → passive checks → factions → companions → admin interface (RP Tester + Workshop). No Unity.

**Phase 1:** Unity vertical slice. One scene, one NPC, one combat encounter, character creation, FishNet host-only, basic traversal. Exit: 60–90 min RP session with visible NPC

body language.

**Phase 2:** Three original worlds. Traversal controllers, economy, items, flora/fauna, crafting, narrative director, canon mutation, companion combat AI, Narrative Control Panel, DM Workshop. Multiplayer testing → assess Postgres migration.

**Phase 3:** Four fandom AU worlds. Canon name gating, per-world ability mappings, Tier 2 access enforcement.

**Phase 4:** Polish. TTS (ADR), multi-NPC scenes, optional grid combat, extended EAS, housing, equipment repair, companion reunion templates.

**Phase 5:** AU world ability mappings, extended content.

## 16. When Uncertain, Ask

Request clarification before proceeding for: schema changes, new service integrations, relay-as-source-of-truth changes, multi-world impacts, trust architecture changes, new dependencies, mixing RP/quick-chat, canon mutation procedure changes, new traversal types, economy changes, WebSocket protocol changes, error schema changes, condition/death mechanics, companion combat behaviour, faction propagation rules, workshop access changes, plot beat schema changes.

*Last updated: May 2026. Revision 5. CLAUDE.md is canonical for architecture and conventions. Design documents in docs/ are canonical for subsystem specifications. If either disagrees with code, the code is wrong. Update when architecture, schemas, or conventions change.*